

Moteur 2 — Le moteur à l'œuvre et nos données BTCUSDT

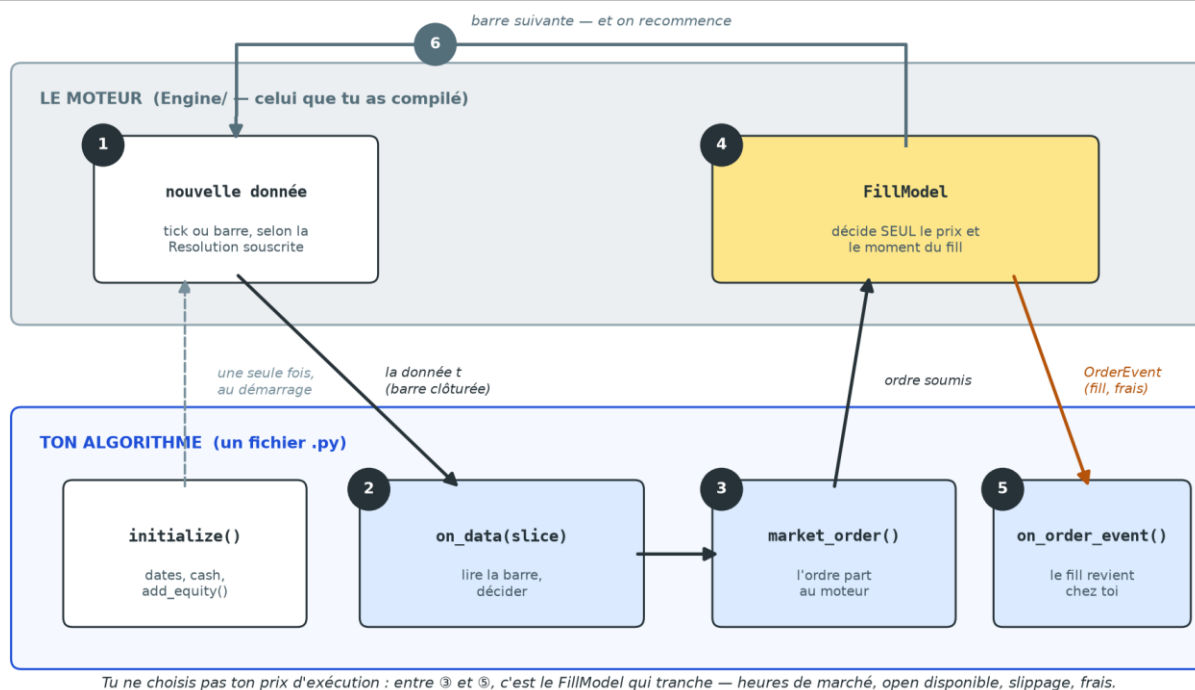
Deux fondations dans cette leçon, qui n'en font qu'une : comprendre **ce que le moteur fait vraiment** (boucle événementielle, ordres, fills — mesurés, pas supposés), puis lui faire consommer **nos données** BTCUSDT à la place du SPY d'exemple. À la fin, LEAN rejoue nos 4 344 barres horaires sans en perdre une, et on sait — preuves à l'appui — à quel prix et à quel instant un ordre est rempli.

La boucle événementielle

Dans un backtester **événementiel**, ton code ne « parcourt » pas les données : le moteur les **pousse**, une à une, dans l'ordre chronologique, et ton algorithme **réagit**. La donnée brute, c'est le **tick** (chaque transaction) ; une barre OHLCV n'est qu'une agrégation. La Resolution souscrite (TICK, SECOND, MINUTE, HOUR, DAILY) décide de ce que le moteur pousse dans `on_data`.

Un algorithme, c'est deux méthodes obligatoires et deux d'observation :

Méthode	Appelée...	Rôle
<code>initialize()</code>	une seule fois, avant la 1 ^{re} barre	déclarer le monde : dates, capital, instruments
<code>on_data(slice)</code>	à chaque barre reçue	lire, décider, passer des ordres
<code>on_order_event(event)</code>	à chaque événement d'ordre	voir les soumissions et les fills
<code>on_end_of_algorithm()</code>	une fois, à la fin	bilan, logs de clôture



La boucle événementielle de LEAN : le flux de données pousse chaque donnée — tick ou barre selon la Resolution souscrite — vers `on_data` ; les ordres partent vers le moteur ; le FillModel décide du prix et du moment du fill, qui revient dans `on_order_event`.

Figure — Le moteur (gris) pousse les barres ; ton code (bleu) réagit ; le FillModel (jaune) décide **seul** du prix et du moment du fill. (Script : `fig_boucle_evenementielle.py`)

Le point crucial : entre ton `market_order()` et le fill, il y a le **FillModel** — le composant du moteur qui simule la réalité du marché. Tu ne choisis pas ton prix d'exécution, tu le **subis**, comme en réel. Deux détails du contrat des barres serviront sans cesse : `bar.time = ouverture`, `bar.end_time = clôture` — et `on_data` est appelé à `end_time`, quand la barre est **finie**. On décide toujours sur du passé.

Premier algorithme

Nos algos vivent **hors du dépôt cloné**, dans un dossier `algorithms/` à côté de `lean/` — le clone reste intact. Celui-ci (`anatomie.py`) est volontairement bavard : il journalise chaque barre, chaque événement d'ordre, un bilan final. Cinq barres journalières de SPY suffisent pour voir vivre le cycle complet :

```
# Anatomie — notre premier algorithme, écrit à la main.
# But : voir vivre le cycle événementiel (initialize → on_data → ordre → fill).
from AlgorithmImports import *

class Anatomie(QCAlgorithm):

    def initialize(self):
        """Appelé UNE fois, avant la première donnée : on déclare le monde."""
        self.set_start_date(2013, 10, 7)
        self.set_end_date(2013, 10, 11)
        self.set_cash(100_000)
        # add_equity abonne l'algo aux barres SPY ; on garde le Symbol
        (l'identifiant interne)
        self.spy = self.add_equity("SPY", Resolution.DAILY).symbol
        self.bars_seen = 0

    def on_data(self, data: Slice):
        """Appelé À CHAQUE barre reçue : c'est ici qu'on vit l'événementiel."""
        if self.spy not in data.bars:          # prudence : pas de barre SPY dans ce
Slice
            return
        bar = data.bars[self.spy]
        self.bars_seen += 1
        # bar.time = OUVERTURE de la barre ; bar.end_time = sa CLÔTURE (= l'heure
de l'événement)
        self.log(f"barre {self.bars_seen} | {bar.time:%Y-%m-%d %H:%M} -> "
                f"{bar.end_time:%H:%M} | O={bar.open} C={bar.close}")
        if not self.portfolio.invested:
            self.market_order(self.spy, 100)    # ordre au marché : 100 actions

    def on_order_event(self, event: OrderEvent):
        """Chaque événement d'ordre (soumis, rempli...) : notre boîte noire des
fills."""
        self.log(f"ORDRE -> {event}")

    def on_end_of_algorithm(self):
        self.log(f"Fin : {self.bars_seen} barres vues, "
                f"équité finale = {self.portfolio.total_portfolio_value:.2f} $")
```

Lancement — mêmes variables d'environnement qu'en Moteur 1, plus `--algorithm-location` qui pointe vers **ton** fichier (le chemin relatif compte 4 niveaux de remontée depuis `bin/Release`) :

```
cd lean/Launcher/bin/Release
$env:PYTHONNET_PYDLL = "C:\Users\<toi>\anaconda3\envs\backtesting\python311.dll"
$env:PYTHONHOME      = "C:\Users\<toi>\anaconda3\envs\backtesting"
```

```
dotnet QuantConnect.Lean.Launcher.dll `
  --algorithm-language Python `
  --algorithm-type-name Anatomie `
  --algorithm-location "../../../../../algorithms/anatomie.py"
Attendu : Total Orders 1, End Equity 100245.42.
```

Ce que le moteur fait de nos ordres — mesuré

Le journal (Anatomie-log.txt, voir plus bas) raconte tout :

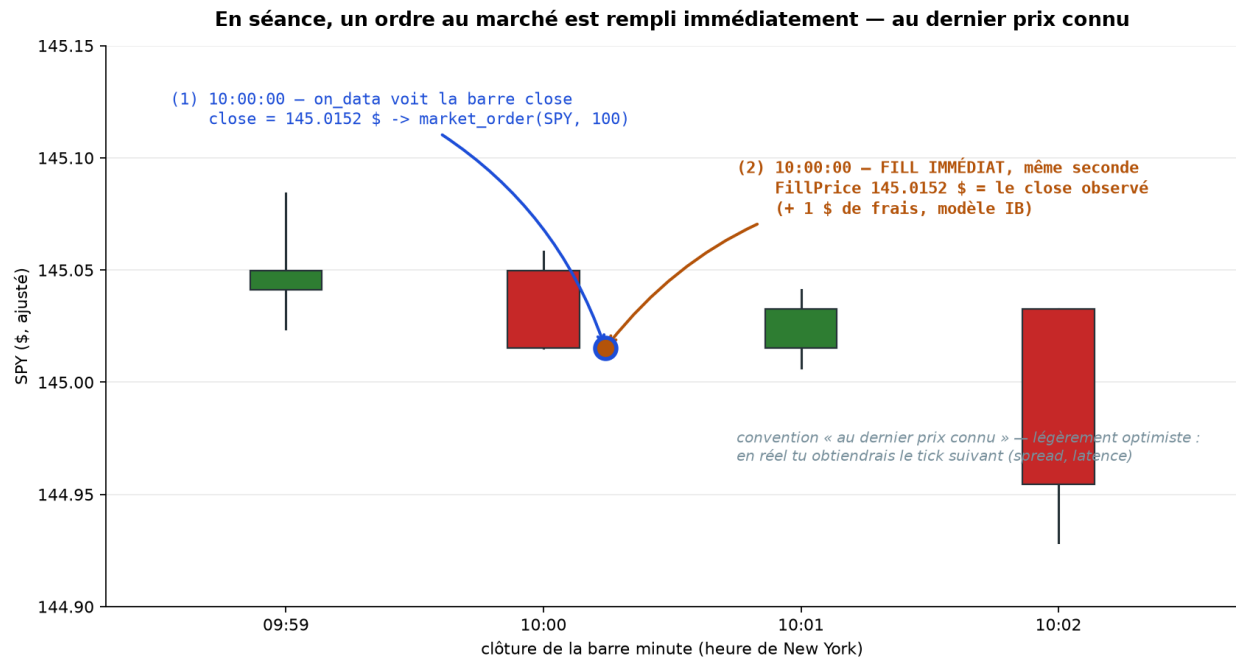
```
2013-10-07 16:00:00 Warning: market orders submitted while the market is closed are
                        automatically converted into MarketOnOpen orders to fill at the
next market open.
2013-10-07 16:00:00 barre 1 | 2013-10-07 09:30 -> 16:00 | O=144.764... C=144.755...
2013-10-07 16:00:00 ORDRE -> ... Status: Submitted
2013-10-08 16:00:00 ORDRE -> ... Status: Filled  FillPrice: $144.7817 OrderFee: 1
USD
```

Quatre faits dans dix lignes :

1. **on_data est appelé à la clôture** : la barre couvre 09:30 → 16:00, le log est horodaté 16:00.
2. **Notre ordre est parti bourse fermée** — inévitable en résolution journalière (la barre se clôt à la fermeture). Le moteur le **convertit en MarketOnOpen** et le remplit **le lendemain à l'open** : 144.7817 = l'open exact de la barre 2. C'est le cas particulier.
3. **OrderFee: 1 USD** : on n'a rien configuré, LEAN applique son modèle de frais par défaut (Interactive Brokers, minimum 1 \$/ordre). **Un backtest a toujours un modèle de frais — le tien ou celui par défaut.**
4. Les OrderEvent sont horodatés en **UTC** (20:00 = 16:00 New York) : premier contact avec le réflexe « quelle horloge lis-je ? ».

Et la **règle générale**, marché ouvert ? Deuxième expérience (anatomie_minute.py, résolution minute, un seul ordre à 10:00 en pleine séance) — résultat mesuré :

```
10:00:00 barre 09:59:00 -> 10:00:00 O=145.0498 ... C=145.015173352
10:00:00 ORDRE -> ... Status: Filled  FillPrice: $145.0152
```



Quatre barres minute SPY autour de 10:00. À 10:00:00, on_data voit le close de la barre et émet market_order ; le fill a lieu la même seconde, au même prix — le dernier prix connu.

Figure — La règle générale, mesurée : décision et fill au **même instant**, au **même prix** (le close observé).
(Script : fig_fill_en_seance.py)

En séance, un ordre au marché est rempli immédiatement, au dernier prix que le moteur connaît — le close de la barre qui a déclenché la décision. Pas la barre suivante, pas son open. Convention **légèrement optimiste** (en réel : tick suivant, spread, latence) — limite assumée, qui resservira à la réconciliation des moteurs. Corollaire mesuré : un signal émis sur la **dernière barre** d'un backtest n'a plus de barre pour être exécuté — c'est un **signal fantôme** (l'ordre reste Submitted, jamais Filled).

Lire le terminal et les fichiers

La console mélange trois familles de lignes (TRACE:: = le moteur raconte sa vie, ignorable ; TRACE:: Debug: = messages importants ; TRACE:: Log: = tes self.log()) et **trois horloges** : la murale (quand la ligne s'imprime — aujourd'hui), la simulée (le présent du backtest, fuseau du marché) et l'UTC des OrderEvents.

Surprise mesurée : les lignes de la console ne sont **pas dans l'ordre** — les self.log() partent dans une file asynchrone, vidée à l'arrêt sur les runs courts. **La console n'est pas une référence chronologique**. La référence, ce sont les quatre fichiers écrits à côté du Launcher :

Fichier	Usage
<Classe>-log.txt	LE journal à lire — chronologie simulée propre, sans bruit moteur
<Classe>.json	séries complètes (équité...) — la matière des figures
<Classe>-summary.json	statistiques finales en JSON
<Classe>-order-events.json	chaque fill en pleine précision — l'outil d'audit

Détail qui compte : le log affiche FillPrice: \$144.7817 (arrondi), l'order-events contient 144.78172417. Pour auditer un fill au centime, le JSON fait foi.

Sous le capot

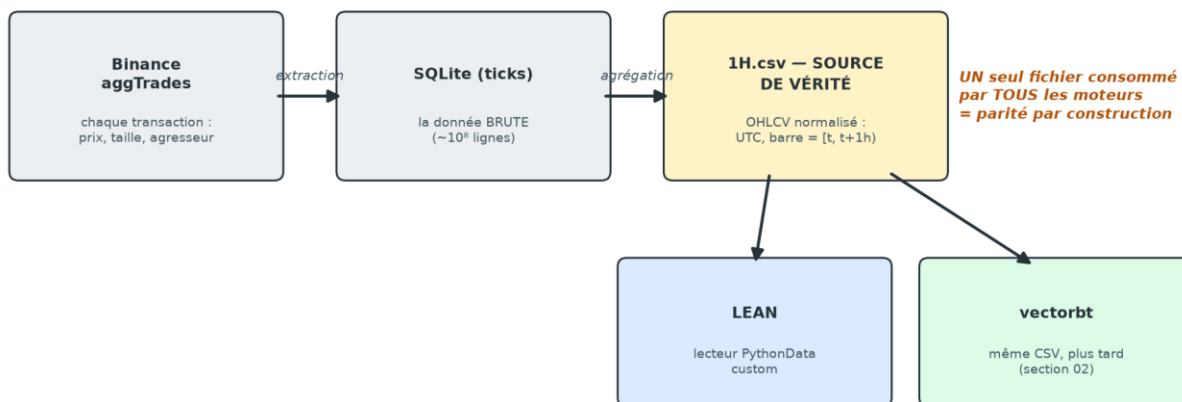
Le bénéfice d'avoir compilé les sources : au lieu de croire cette leçon sur parole, on lit les lignes qui ont produit nos journaux (depuis la racine du clone) :

```
Select-String -Path Engine/AlgorithmManager.cs -Pattern "algorithm.OnData\"
Select-String -Path Common/Orders/Fills/FillModel.cs -Pattern "Fills at the last
traded price"
Select-String -Path Algorithm/QCAlgorithm.Trading.cs -Pattern "converted into
MarketOnOpen"
```

AlgorithmManager.cs contient la ligne de la boucle (algorithm.OnData(...)); FillModel.cs documente la règle générale noir sur blanc (« Fills at the last traded price », avec le garde IsExchangeOpen et un contrôle de données périmées) ; et QCAlgorithm.Trading.cs porte le Warning exact de notre journal. La décision « à quel prix mon ordre est-il rempli ? » vit dans Common/Orders/Fills/FillModel.cs — retiens l'adresse.

Nos données : la source de vérité

Fini le SPY d'exemple. Notre matière première : **BTCUSDT futures perpétuels (marge USDT)** — des ticks bruts Binance stockés en SQLite, agrégés en barres OHLCV : le fichier 1H.csv. Ce fichier a un statut spécial : c'est la **source de vérité**, que LEAN et (plus tard) vectorbt consommeront tel quel. Un seul fichier pour tous les moteurs = tout écart futur de backtest sera imputable aux **moteurs**, pas aux données. (Le format natif LEAN reste un upgrade « live-ready » optionnel, bien plus tard.)



Si les deux moteurs lisent le même fichier, tout écart de résultat vient des moteurs — pas des données. C'est ça, une source de vérité.

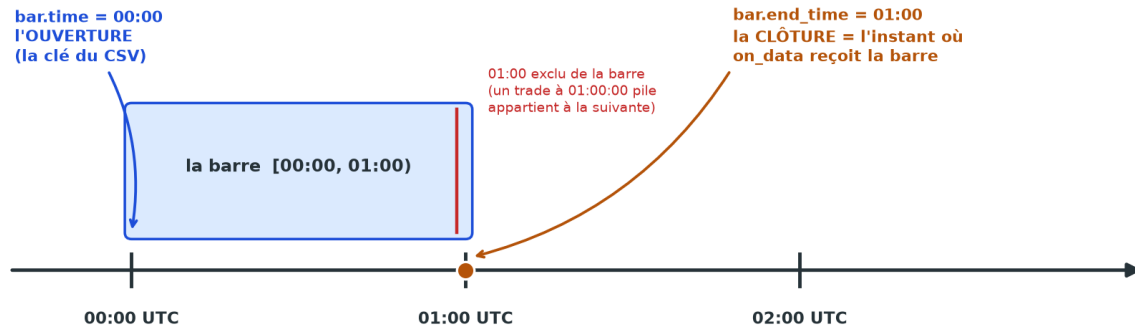
Pipeline des données : Binance aggTrades vers SQLite (ticks bruts), agrégés en 1H.csv la source de vérité OHLCV normalisée UTC, consommée à la fois par LEAN et plus tard par vectorbt.

Figure — Un seul fichier consommé par tous les moteurs. (Script : fig_source_de_verite.py)

On ne branche jamais un fichier qu'on n'a pas regardé :

```
Get-Content H:\Crypto\historique\ohlcv\BTCUSDT-um\1H.csv -TotalCount 3
time,open,high,low,close,volume,buy_volume,trades
2026-01-01 00:00:00+00:00,87608.3,87819.0,87600.0,87773.4,1571.03...,914.09...,14848
2026-01-01 01:00:00+00:00,87773.4,88021.4,87773.3,87916.5,2441.13...,1255.92...,20444
```

Le contrat, à connaître par cœur : `time = ouverture` de la barre, en **UTC explicite** ; `barre = [t, t+1h)` ; volumes en **BTC** ; `buy_volume` = part du volume à l'initiative des acheteurs (la graine des futures stratégies d'order-flow).



△ PIÈGE : sans `set_time_zone(UTC)` + `add_data(..., TimeZones.UTC)`, LEAN interprète ces horodatages naïfs comme de l'heure de NEW YORK — tout glisse de 4-5 h et le trou de l'heure d'été avale une barre en silence.

Le contrat d'une barre : `bar.time` = l'ouverture (la clé du CSV), `bar.end_time` = la clôture, l'instant où `on_data` reçoit la barre ; la borne haute est exclue.

Figure — La barre $[t, t+1h)$. (Script : `fig_contrat_barre.py`)

Le lecteur PythonData

Deux morceaux dans `donnees.py` : le **lecteur** (LEAN appelle `reader` pour chaque ligne du CSV et attend une barre, ou `None` pour ignorer) et un **algorithme de contrôle** qui ne trade pas — il vérifie le contrat, avec des bornes lues **dans le fichier lui-même** (l'historique s'étend : on ne fige jamais de date en dur).

```
# Brancher NOS données BTCUSDT (futures perp, marge USDT) dans LEAN.
# Pas de trading ici : on vérifie le CONTRAT de données (UTC, bornes, valeurs).
from AlgorithmImports import *
```

```
DATA_FILE = "H:/Crypto/historique/ohlcv/BTCUSDT-um/1H.csv"    # ← adapte à ta
machine
```

```
class BtcUsdtHourly(PythonData):
    """Lecteur custom : une ligne du CSV -> une barre LEAN."""

    def get_source(self, config, date, is_live):
        # LOCAL_FILE : LEAN lit notre fichier tel quel, où qu'il soit sur le disque
        return SubscriptionDataSource(DATA_FILE,
SubscriptionTransportMedium.LOCAL_FILE)

    def reader(self, config, line, date, is_live):
        if not line or not line[0].isdigit():          # ignorer l'en-tête
            return None
        cols = line.split(",")
        bar = BtcUsdtHourly()
        bar.symbol = config.symbol
        # '2026-01-01 00:00:00+00:00' -> datetime naïf ; add_data(...,
TimeZones.UTC)
```

```

!))
    # dira à LEAN de l'interpréter en UTC (PIÈGE : défaut = heure de New York
    t_open = datetime.strptime(cols[0][:19], "%Y-%m-%d %H:%M:%S")
    bar.time = t_open # ouverture de la barre
    bar.end_time = t_open + timedelta(hours=1) # clôture = moment où on_data
la reçoit
    bar.value = float(cols[4]) # close = valeur de référence
(obligatoire)
    bar["open"] = float(cols[1])
    bar["high"] = float(cols[2])
    bar["low"] = float(cols[3])
    bar["close"] = float(cols[4])
    bar["volume"] = float(cols[5]) # en BTC
    return bar

```

```

class Donnees(QCAlgorithm):

```

```

    def initialize(self):
        # Bornes ADAPTATIVES : lues dans le CSV lui-même
        with open(DATA_FILE) as f:
            rows = f.read().splitlines()
            premier = datetime.strptime(rows[1][:19], "%Y-%m-%d %H:%M:%S")
            dernier = datetime.strptime(rows[-1][:19], "%Y-%m-%d %H:%M:%S")
            self.set_start_date(premier.year, premier.month, premier.day)
            self.set_end_date(dernier.year, dernier.month, dernier.day)
            self.set_cash(100_000)

        # LE point critique : TOUT en UTC – l'horloge de l'algo ET la donnée custom
        self.set_time_zone(TimeZones.UTC)
        self.btc = self.add_data(BtcUsdtHourly, "BTCUSDT", Resolution.HOUR,
                                TimeZones.UTC).symbol

        self.attendu_premier, self.attendu_dernier = premier, dernier
        self.bars = 0
        self.premiere_barre = None
        self.derniere_barre = None

    def on_data(self, data: Slice):
        if self.btc not in data:
            return
        bar = data[self.btc]
        self.bars += 1
        if self.bars <= 3: # inspection visuelle du contrat
            self.log(f"barre {self.bars} | {bar.time:%Y-%m-%d %H:%M} UTC -> "
                    f"{bar.end_time:%H:%M} | O={bar['open']} C={bar['close']} "
                    f"V={bar['volume']:.1f} BTC")
        if self.premiere_barre is None:
            self.premiere_barre = bar.time
        self.derniere_barre = bar.time

    def on_end_of_algorithm(self):
        ok_debut = self.premiere_barre == self.attendu_premier
        ok_fin = self.derniere_barre == self.attendu_dernier
        self.log(f"Bilan : {self.bars} barres | première {self.premiere_barre:%Y-%

```

```
%m-%d %H:%M} "
                f"(= CSV ? {ok_debut}) | dernière {self.derniere_barre:%Y-%m-%d
%H:%M} "
                f"(= CSV ? {ok_fin}))")
Lancement(--algorithm-type-name Donnees --algorithm-location
"../../../../algorithms/donnees.py"), puis Get-Content Donnees-log.txt :
barre 1 | 2026-01-01 00:00 UTC -> 01:00 | O=87608.3 C=87773.4 V=1571.0 BTC
barre 2 | 2026-01-01 01:00 UTC -> 02:00 | O=87773.4 C=87916.5 V=2441.1 BTC
barre 3 | 2026-01-01 02:00 UTC -> 03:00 | O=87916.6 C=87884.6 V=1895.8 BTC
Bilan : 4344 barres | première 2026-01-01 00:00 (= CSV ? True) | dernière 2026-06-
30 23:00 (= CSV ? True)
```

La barre 1 est reçue à sa clôture (01:00), les valeurs collent au CSV, **aucune barre perdue** de bout en bout. (Tes chiffres différeront — l'historique grandit ; ce qui doit correspondre, ce sont les True.)

En fin de run, une erreur `BacktestingResultHandler.SendFinalResult(): ... Sequence contains no elements` peut apparaître : artefact **bénin** de nos runs 100 % donnée custom, sans benchmark (mesuré : avec ou sans ordres). Logs, statistiques et JSON sont intacts.

La preuve du fuseau : une barre avalée en silence

Le code impose `set_time_zone(UTC)` et `add_data(..., TimeZones.UTC)`. Sur parole ? Non — l'expérience `donnees_fuseau.py` **retire** ces deux réglages et ajoute un détecteur de trous (`bar.time - derniere_barre != 1h`). Résultat mesuré :

```
TROU ! saut de 2026-03-07 23:00 a 2026-03-08 01:00
TROU ! saut de 2026-03-08 01:00 a 2026-03-08 03:00
TROU ! saut de 2026-03-08 03:00 a 2026-03-08 03:00
Bilan : 4343 barres | première ... (= CSV ? True) | dernière ... (= CSV ? True)
```

Sans fuseau explicite, LEAN interprète nos horodatages naïfs en **heure de New York**. Tout glisse de 5 h (l'écart NY↔UTC d'hiver ; 4 h l'été) de façon cohérente — rien ne choque... jusqu'au **8 mars 2026, 02:00** : l'heure qui **n'existe pas** à New York (passage à l'heure d'été). Une barre **avalée** (4343 au lieu de 4344), des sauts, un doublon d'horodatage — aucune erreur, aucun warning, et un bilan qui affiche fièrement True aux deux bornes. Des données silencieusement fausses deux fois par an : voilà pourquoi les deux réglages UTC ne sont pas négociables.

Ce qu'il faut retenir

- `initialize` (une fois) + `on_data` (chaque barre, appelée à la **clôture** — on décide sur du passé). Entre l'ordre et le fill : le **FillModel** tranche, pas toi.
- **Sémantique des fills, mesurée** : en séance = immédiat, **au dernier prix connu** (le close observé — convention optimiste assumée) ; bourse fermée (dont toute résolution journalière) = converti `MarketOnOpen`, rempli à l'open suivant. Et il y a **toujours** un modèle de frais (défaut : IB, min 1 \$/ordre).
- La console n'est pas chronologique ; la référence = `<Classe>-log.txt`, l'audit des fills = `-order-events.json` (pleine précision).
- **Une source de vérité** : le même CSV pour tous les moteurs — parité par construction. Contrat : `time` = ouverture UTC, barre `[t, t+1h)`, `bar.value` obligatoire, jamais de NaN (crash Decimal).
- **TOUT en UTC, explicitement** (les deux réglages) : le défaut New York avale une barre au changement d'heure — en silence.

Précédent : [Moteur 1 — Installation](#) · Suite : [Stratégie 1 — Buy & Hold et le sol de vérité](#), premier vrai backtest sur nos données : frais taker 0,04 %, BTC fractionnaire, et l'équité recalculée à la main au centime près.